UNIVERSITÀ DEGLI STUDI DI GENOVA

DIBRIS

DEPARTMENT OF INFORMATICS,
BIOENGINEERING, ROBOTICS AND SYSTEM ENGINEERING

ARTIFICIAL INTELLIGENCE

Assignment

Automated Guided Vehicles for material transport in mining sites

---

Author:

Joel Topulli
Stefania Germena

Student ID:

S8663382
S7903986

Professor:

Armando Tacchella

January 31, 2026

Contents

1 Abstract **2**

2 Ontology Design and Rationale **2**
2.1 Core Classes and Hierarchy . . . 2
2.2 AVG classification Strategy . . . 3
2.3 Object Properties and Relationships . . . 3
2.3.1 Functional Properties . . . 3
2.3.2 Non-Functional Properties . . . 3
2.3.3 Asymmetric and Irreflexive Properties . . . 4

3 Ontology Validation and Reasoning **4**
3.1 Instance Modeling . . . 4
3.2 Reasoning and Logic Queries . . . 4
3.3 Consistency Checking . . . 5

4 Design Decisions and Simplifications **5**
4.1 Scope Limitations . . . 5
4.2 Load Weight vs. Payload Capacity Constraint . . . 5
4.3 State Modeling . . . 5

5 Previous Errors and Resolutions **6**
5.1 Structural Integrity and Domain Reconciliation . . . 6
5.2 Classification Consistency and Data Completeness . . . 6
5.3 Datatype Precision and Query Validation . . . 6

6 Conclusion **7**
6.1 Key Learning Outcomes . . . 7
6.2 Design Trade-offs and Engineering Decisions . . . 7
6.3 Methodological Insights . . . 8
6.4 Closing Remarks . . . 8

References **9**

1 Abstract

This report presents the design and ontology modeling of an Automated Guided Vehicle (AGV) system for underground mining applications. The ontology captures the essential domain knowledge required for AGV management, including but not limited to vehicle specifications, navigation, infrastructure, material handling, and operational zones. The focus is on identifying and formalizing the relationships between sites, zones, loads, tasks, AGV identification, and navigation constraints. The proposed ontology enables reasoning about AGV capabilities, current state, and operational constraints that would allow automated decision making in a real-world scenario. We demonstrate the ontology's utility through DL queries and discuss the motivation behind the design choices that made this domain possible.

2 Ontology Design and Rationale

2.1 Core Classes and Hierarchy

The ontology is structured around nine top-level disjoint classes, each representing a fundamental domain concept. This disjoint-ness ensures a clear separation of entities and prevents logical inconsistencies within the ontology. The choice of these classes was heavily influenced by the existing literature, especially *Autonomous Driving of Trackless Transport Vehicles* [2].

Class	Design Rationale
AGV	Central entity representing autonomous vehicles. Classified into *Large*, *Medium*, and *Small* based on payload capacity, speed, and width constraints.
Site	Disjoint locations (*Mine*, *Excavation Site*, *Metal Extraction Site*, *Disposal Site*) ensuring AGVs have unambiguous spatial assignment.
Zone	Operational areas within sites (*Loading*, *Dumping*, *Charging*, *Waiting*, *Restricted*). Declared disjoint to prevent simultaneous presence in conflicting zones.
Load	Material carried (*Ore*, *Waste*, *Elements*, *Empty*). Disjoint to enforce a single load-type constraint, preventing logical contradictions.
Task	Current activity (*Loading*, *Unloading*, *Transit*, *Reversing*, *Stopping*, *Charging*). Disjoint to ensure only one task is executed at a time.
Infrastructure	Physical environment (*Tunnel*, *Level*, *Intersection*, *Steepness Gradient*). Required for traffic management and optimal pathing.
Electronic Components	Hardware components (*Camera*, *Sensors*, *MicroController*, *Lights*, *Speakers*). Enables modeling of AGV capabilities and operational requirements.
AGV Specifications	Physical characteristics. The only subclass is wheel types (*Grippy*, *Normal*, *Track*), with size attributes used to determine terrain suitability.
Load Specifications	Load-related properties such as the *weight*, measured by onboard sensors.

Table 1: Top-level ontology classes and their design rationale

It is important to emphasize that the classes and their respective subclasses are not intended to represent an exhaustive catalog of the AGV domain. This a deliberate design decision as our primary objective was to establish a logically sound architectural *skeleton* [1], which can be modified and expanded without issues.

2.2 AVG classification Strategy

Being the cornerstone of our project, AGVs are automatically classified, into distinct size categories based on three data properties, by using the built in reasoner in *Protégé*.

Class	Payload (kg)	Max Speed (m/s)	Width (m)
Large	10,000 – 70,000	0.1 – 0.5	≥ 3.0
Medium	1,000 – 5,000	≤ 1.7	≤ 1.2
Small	200 – 500	0.5 – 1.0	≤ 1.0

Table 2: AGV size classification based on data properties

This classification enables automatic inference of AGV capabilities based on manufacturer specifications. Hence, when you are defining the individuals you won't need to populate the classes or sub-classes with instances. However, you need to be extra careful when you define its data properties.

2.3 Object Properties and Relationships

The ontology employs carefully designed object properties with specific characteristics to ensure logical consistency and enable effective reasoning.

2.3.1 Functional Properties

Several properties are declared as functional, enforcing cardinality constraints that are critical for operational validity:

- *hasLoad* – An AGV can carry only one type of load at a time.
- *doingTask* – An AGV can perform only one task simultaneously.
- *in_Site* – An AGV can physically be located at only one site.
- *in_Zone* – An AGV can occupy only one operational zone.
- *AGV_specif* – Each AGV has a single set of specifications.

These functional constraints were validated through inconsistency testing. For example, assigning both *hasElements* (iron) and *hasWaste* (mud) to the same AGV individual (*CAT*) correctly triggers a reasoner error, thereby confirming enforcement of the disjoint load-type constraint.

2.3.2 Non-Functional Properties

In contrast, object properties related to electronic components (*hasCamera*, *hasSensor*, *hasMicroController*, *hasLights*, *hasSpeaker*) are intentionally not declared as functional. This design choice reflects real-world AGV architectures, which commonly include multiple cameras (e.g., front and rear), multiple sensors (LiDAR, infrared, weight, tilt, accelerometer), and potentially several lights or speakers.

Initially, these properties were modeled as functional; however, this introduced conflicts with the AGV class definition requiring:

$$\exists\, hasCamera.Camera,$$

which enforces the presence of at least one camera. Declaring the property as functional would restrict each AGV to exactly one camera, contradicting the dual-camera navigation design. By removing the functional constraint while retaining the existential restriction, the ontology permits multiple components while still ensuring minimum hardware requirements are satisfied.

2.3.3 Asymmetric and Irreflexive Properties

All major object properties are additionally declared as *asymmetric* and *irreflexive* to prevent illogical inferences. For example, if:

$$AGV_1 \ in_Site \ Mine_1,$$

it cannot be inferred that:

$$Mine_1 \ in_Site \ AGV_1$$

(asymmetry), nor that:

$$AGV_1 \ in_Site \ AGV_1$$

(irreflexivity). These constraints preserve semantic coherence and prevent wrong reasoning outcomes.

3 Ontology Validation and Reasoning

3.1 Instance Modeling

To validate the ontology's logical consistency and reasoning capabilities, it was populated with representative instances of real AGV models.

Individual	Type	Specifications	Location & Task
BelAZ	Large	20,000 kg payload, 0.2 m/s max speed, 4.0 m width	Excavation Site, Waiting Zone, carrying copper
Komatsu	Medium	4,000 kg payload, 1.5 m/s max speed, 1.1 m width	Mine, Loading Zone, carrying iron ore
CAT	Small	300 kg payload, 0.8 m/s max speed, 0.9 m width	Disposal Site, Dumping Zone, carrying waste mud

Table 3: Representative AGV individuals used for ontology validation

3.2 Reasoning and Logic Queries

Logic queries were executed to demonstrate the ontology's reasoning capabilities. Consider the following example query:

$$AGV \sqcap \exists carriesLoad.\Big(Ore \sqcap \exists load_specif.\big(load_weight \sqcap \exists weight_value.\texttt{xsd:decimal}\big)\Big)^{1}$$

This query retrieves all AGVs currently transporting iron ore that possess specific weight measurements. The reasoner correctly identifies individuals such as *Komatsu* as matching results based on their assigned object and data property chains.

The successful execution of this query validates several critical dimensions of the ontology:

- The `carriesLoad` object property correctly links the mobile robot to its physical cargo.
- The `load_specif` object property successfully bridges the material individual to its technical data sheet.
- The `weight_value` data property accurately processes numerical `xsd:decimal` values for quantitative reasoning.
- The nested conjunction of existential restrictions (class intersections) operates correctly across multiple hierarchical levels.

Beyond theoretical validation, this query establishes a functional interface for real-time control integration. Retrieving specific load classifications and mass data allows the AGV to execute pathfinding optimization and maintain dynamic velocity constraints. This result is required to mitigate operational risks and ensure the structural safety of the mining environment in a real-world scenario.

[1] The actual DL Query: `AGV and carriesLoad some (Ore and load_specif some (load_weight and weight_value some xsd:decimal))`

3.3 Consistency Checking

The ontology's disjointness constraints were validated through deliberate inconsistency introduction. When both *hasElements(iron)* and *hasWaste(mud)* were assigned to the *CAT* individual, the reasoner correctly detected an unsatisfiable class, confirming that the Load subclasses are properly disjoint.

This validation is critical because it prevents illogical operational states. An AGV cannot simultaneously transport both valuable ore and waste material; such a condition would violate safety protocols and indicate a system error requiring immediate intervention.

Similar validation was performed for Zone and Site disjointness. The reasoner successfully detected inconsistencies when an AGV was placed in multiple contradictory locations or zones simultaneously, confirming that the spatial constraints are correctly modeled and enforced.

4 Design Decisions and Simplifications

4.1 Scope Limitations

Several aspects of AGV systems were deliberately excluded from the ontology to maintain focus on the core requirements:

- Detailed sensor fusion algorithms and real-time control systems
- Battery management and charging infrastructure specifics
- Maintenance scheduling and fault diagnosis procedures
- Human–machine interface design and operator interaction

These exclusions reflect the ontology's intended purpose: to model the knowledge required for AGV coordination and task assignment, rather than to represent the complete engineering complexity of autonomous vehicle systems. A comprehensive mining automation ontology would require integration with separate ontologies dedicated to these specialized domains.

4.2 Load Weight vs. Payload Capacity Constraint

A notable simplification involves the relationship between load weight and AGV payload capacity. While the ontology includes both *Load_Specifications* (with the *load_weight* subclass) and the AGV data property *hasPayLoadCapacity*, we did not implement a formal constraint ensuring that the load weight never exceeds the payload capacity.

This design decision was made for two primary reasons:

1. **Complexity** — Implementing numeric comparisons in OWL requires SWRL rules or complex property chains, significantly increasing reasoning complexity.
2. **External validation** — Weight constraints are typically enforced by weighbridge systems and load sensors prior to loading, making them operational rather than ontological concerns.

Nevertheless, this limitation is acknowledged. In a production system, such constraints should be implemented within the application layer using appropriate safety interlocks. Even though it does not enforce it automatically, the ontology provides the necessary conceptual foundations such as *load_weight* and *hasPayLoadCapacity*, to support this validation.

4.3 State Modeling

The ontology includes instances of AGV states (*state_idle*, *state_moving*, *state_loaded*, *state_blocked*, *state_faulty*, *state_overLoaded*, *state_waiting*), but does not formally define a dedicated *State* class hierarchy. This was a pragmatic design decision.

AGV states can largely be inferred from existing properties. For example, an AGV with *doingTask.Transit* is implicitly in a moving state, while an AGV with *hasLoad.Empty* and *doingTask.Stopping* is implicitly idle. Introducing a separate *State* class with disjoint subclasses would therefore increase modeling complexity without providing substantial reasoning benefits.

Furthermore, operational state can be inherently ambiguous. An AGV may be simultaneously loaded (carrying cargo) and blocked (waiting for path clearance). Modeling all possible state combinations explicitly would lead to a combinatorial explosion of subclasses.

Instead, the ontology relies on the combined interpretation of *Task*, *Load*, and *Zone* properties to represent the current operational condition of each AGV.

Should explicit state modeling become necessary in future extensions, a *State* class with appropriate subclasses and a functional *hasState* property could be introduced. The existing state individuals would then naturally serve as instances of these state classes.

5 Previous Errors and Resolutions

To ensure the ontology remains a robust framework for robotic reasoning, several critical structural and logical challenges were identified and addressed during development. The following sections detail the primary issues and the engineering rationale behind their resolutions.

5.1 Structural Integrity and Domain Reconciliation

A primary conflict arose concerning the localization of infrastructure. Initially, the *has_Infrastructure* property was erroneously situated within the domain of the *AGV* class, implying a part-of relationship between the mobile robot and its environment. Because the ontology enforces strict disjointness between *AGV* and *Site*, the reasoner identified a logical contradiction: it could not reconcile a mobile agent "owning" static structures such as tunnels or intersections.

To resolve this, the domain of *has_Infrastructure* was reassigned to the *Site* class. This adjustment allows for a more accurate representation, where the *Site* contains the *Infrastructure*, and the *AGV* occupies these spaces through the *in_Site* and *in_Zone* object properties.

5.2 Classification Consistency and Data Completeness

Ensuring that individuals such as *Komatsu* or *BelAZ* are correctly classified into their respective categories, *Small*, *Medium*, or *Large*, required strict adherence to intersection logic. Initially, classification failed because individuals lacked the full set of attributes required by the equivalent class definitions. For instance, a robot cannot be inferred as *Medium* if its *hasWidth* or *hasMaxSpeed* data properties are missing, even if its payload is within the correct range.

By methodically asserting the complete set of functional data properties for each instance, the reasoner successfully performed automatic classification, allowing the robot to "understand" its physical class and capacity relative to the mine's operational constraints.

5.3 Datatype Precision and Query Validation

The integration of quantitative data necessitated a shift from generic literals to specific XML Schema Datatypes (XSD). Early iterations failed to process range-based logic (e.g., $1000 \leq$ payload ≤ 5000) because numerical values were not explicitly typed. By enforcing `xsd:integer` for *hasPayLoadCapacity* and `xsd:decimal` or `xsd:float` for measurements such as *weight_value* and *hasMaxSpeed*, the mathematical validity of the classification was secured.

6 Conclusion

This project demonstrates the practical application of ontology engineering principles learned throughout the course. The AGV mining system served as a domain for exploring fundamental concepts in knowledge representation, reasoning, and semantic modeling using Protégé and OWL.

6.1 Key Learning Outcomes

The development of this ontology provided hands-on experience with several core ontology engineering concepts:

Class Hierarchy Design: The process of identifying and organizing domain concepts into a coherent class structure revealed the importance of balancing granularity with maintainability. Deciding which concepts warrant separate classes versus being represented as properties or individuals required careful analysis of the domain's inherent structure. The nine top-level disjoint classes (AGV, Site, Zone, Load, Task, Infrastructure, Electronic_Components, AGV_Specifications, Load_Specifications) emerged from iterative refinement rather than initial planning, demonstrating that ontology development is inherently exploratory.

Property Characteristics: Understanding when to apply functional, asymmetric, and irreflexive property constraints proved critical. The decision to make *hasLoad* functional while keeping *hasCamera* non-functional illustrates how property characteristics must align with domain semantics. This required moving beyond treating properties as mere attribute holders to understanding them as formal relationships with logical implications.

Disjointness and Reasoning: Declaring subclasses disjoint (e.g., Ore, Waste, Elements, Empty) enables automated consistency checking. The validation test where assigning both *hasElements* and *hasWaste* to the CAT individual triggered a reasoner error demonstrated OWL's capability to detect logical contradictions. This practical experience with consistency checking reinforced the theoretical understanding of closed-world reasoning in description logics.

Equivalent Class Axioms: The automatic classification of AGVs into Large, Medium, and Small categories based on data property ranges showcased OWL's inferencing power. Rather than manually asserting class membership, we defined equivalence conditions that allow the reasoner to derive classifications. This experience illustrated the distinction between asserted and inferred knowledge.

DL Queries: Formulating description logic queries to retrieve specific AGV configurations (e.g., vehicles with specific load types) provided practical experience with the expressive power of description logics. These queries demonstrated how formal ontologies enable precise information retrieval beyond simple keyword matching.

6.2 Design Trade-offs and Engineering Decisions

The ontology development process required making explicit trade-offs between competing concerns:

Expressiveness vs. Computational Tractability: The decision not to implement numeric comparison constraints (load weight $\leq$ payload capacity) exemplifies the balance between what can be modeled and what should be modeled in OWL. While such constraints are theoretically possible through SWRL rules, they introduce computational complexity that may not justify the benefit. This experience reinforced the lesson that ontology engineering requires pragmatic judgment, not just technical knowledge.

Completeness vs. Scope Management: Even though we had external sources, determining the appropriate boundaries for the ontology, such as what to include and what to exclude, was challenging. The decision to model infrastructure elements (Tunnel, Level, Intersection, Steepness_Gradient) but not detailed sensor specifications reflected a judgment about which concepts are essential for the core reasoning tasks versus which are implementation details better handled elsewhere.

Ideal Modeling vs. Practical Implementation: The acknowledgment that sensor requirements are defined in the class structure but not fully enforced through instance population illustrates the difference between conceptual modeling and complete system implementation. Within the scope of this academic exercise, demonstrating understanding of how to structure requirements is more valuable than exhaustively populating every possible instance.

6.3 Methodological Insights

Several methodological lessons emerged from the development process:

Iterative Refinement: The ontology was not designed in a single pass. Initial modeling decisions were revised as understanding deepened. For example, the realization that component properties should be non-functional (allowing multiple cameras and sensors) came from attempting to model specific AGV instances and encountering conflicts with the functional property constraints. This iterative approach mirrors real-world ontology development practices.

Validation Through Instances: Defining specific individuals (BelAZ, Komatsu, CAT, etc.) proved essential for validating the ontology's logical consistency and reasoning capabilities. Without concrete instances, many modeling errors would have remained undetected. This reinforces the principle that ontologies should be developed in conjunction with representative data.

Documentation of Rationale: Throughout this report, we emphasized not just what was modeled, but why particular design choices were made. This practice of documenting design rationale is essential in ontology engineering, where the reasoning behind modeling decisions may not be obvious from examining the OWL file alone. The ontology artifacts (classes, properties, axioms) capture the what; while the documentation captures the why.

6.4 Closing Remarks

The resulting ontology represents a basic, but sound understanding of the factors that need to be considered in autonomous guided vehicles in mining. Not only does it allow for a well defined system, but it also provides a strong foundation for future work as to further expand and create a fully integrated system.

References

[1] N. F. Noy and D. L. McGuinness. *Ontology Development 101: A Guide to Creating Your First Ontology*. Technical Report KSL-01-05. Stanford Knowledge Systems Laboratory, 2001.

[2] Yunjie Sun et al. “Autonomous Driving of Trackless Transport Vehicles: A Case Study in Underground Mines”. In: *Sensors* 25.10 (2025), p. 28.